Color Following

1. Course Content

- Run example programs, the default following color is red. You can press the `r/R` key to enter color selection mode, select a color with the mouse, the screen will lock this color, press the `spacebar` to enter following mode. The car will always ensure the followed object stays in the center of the screen and maintains a distance of 0.4 meters from the object. Press the `q/Q` key to exit the program.

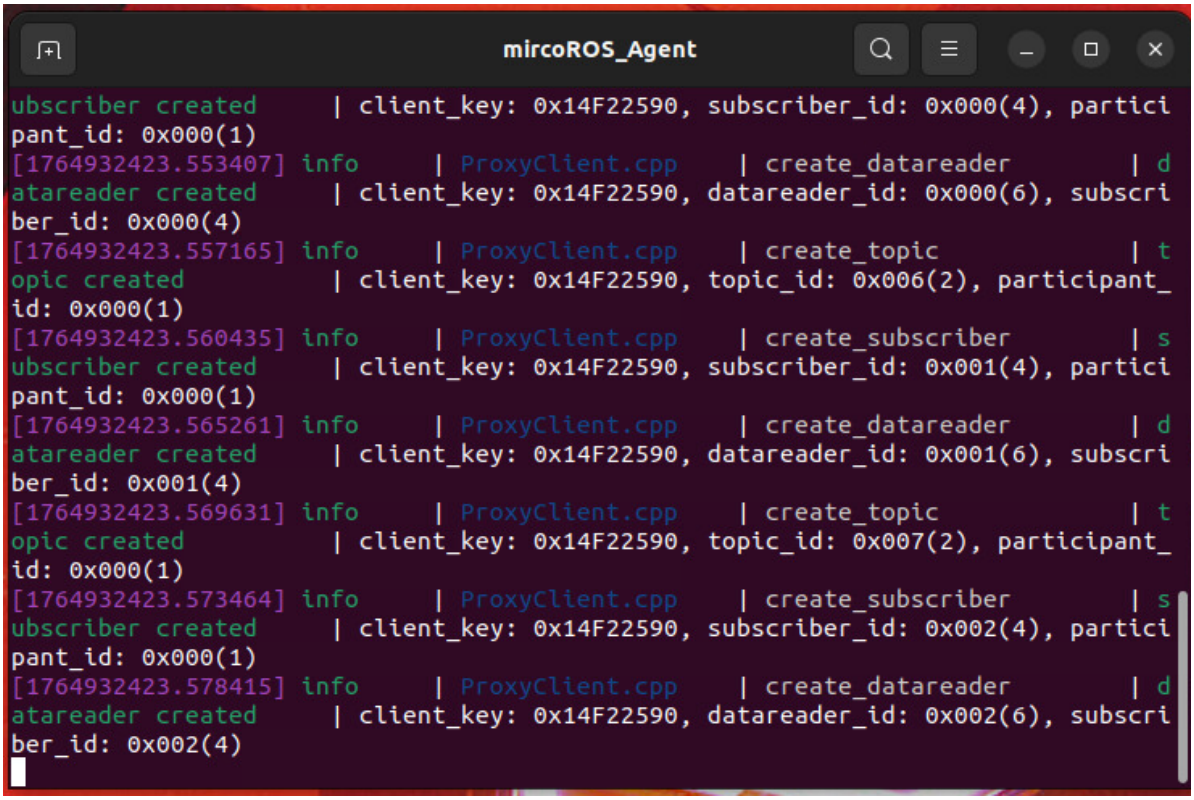
2. Start Agent

Note: If already started, no need to restart

Enter command in the car terminal:

```
start_agent
```

The terminal prints the following information, indicating successful connection



3. Run Case

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

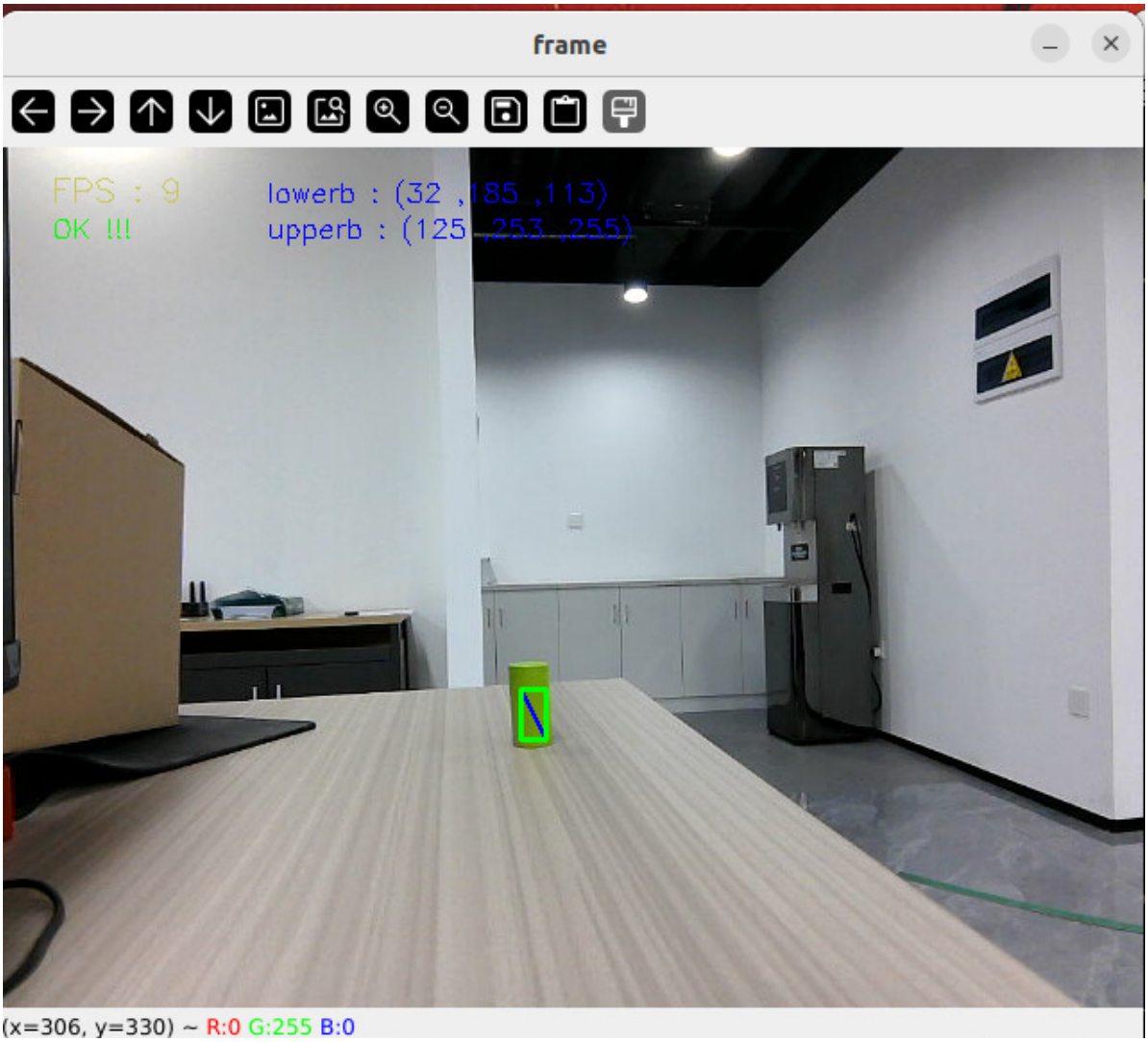
Open a terminal inside the container,

```
exec_m3
```

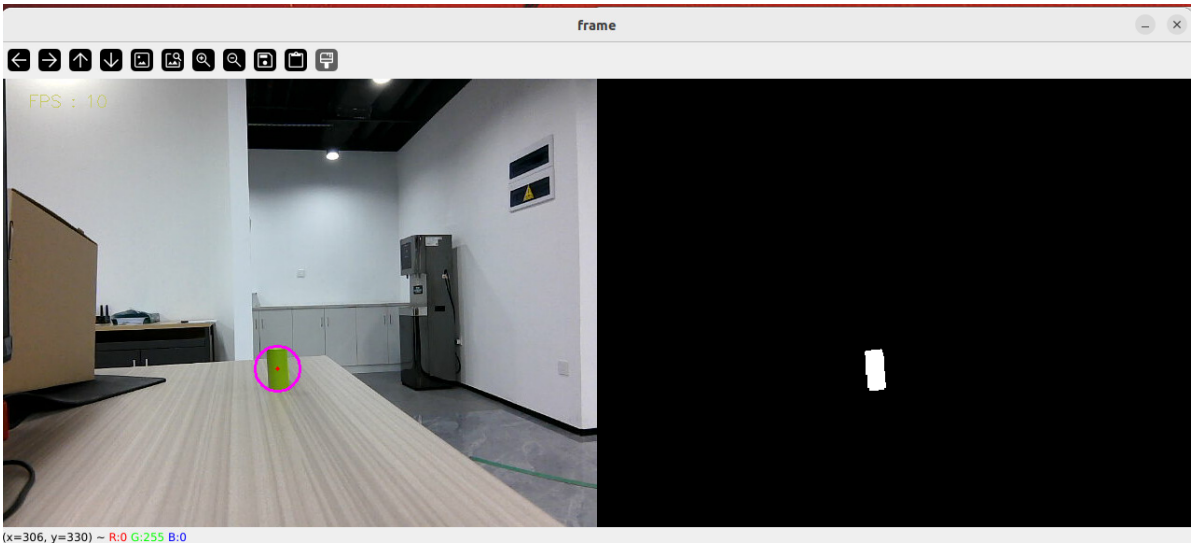
- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=colorTracker
```

- Then press the `r/R` key on the keyboard to enter color selection mode, use the mouse to frame an area (this area can only have one color),



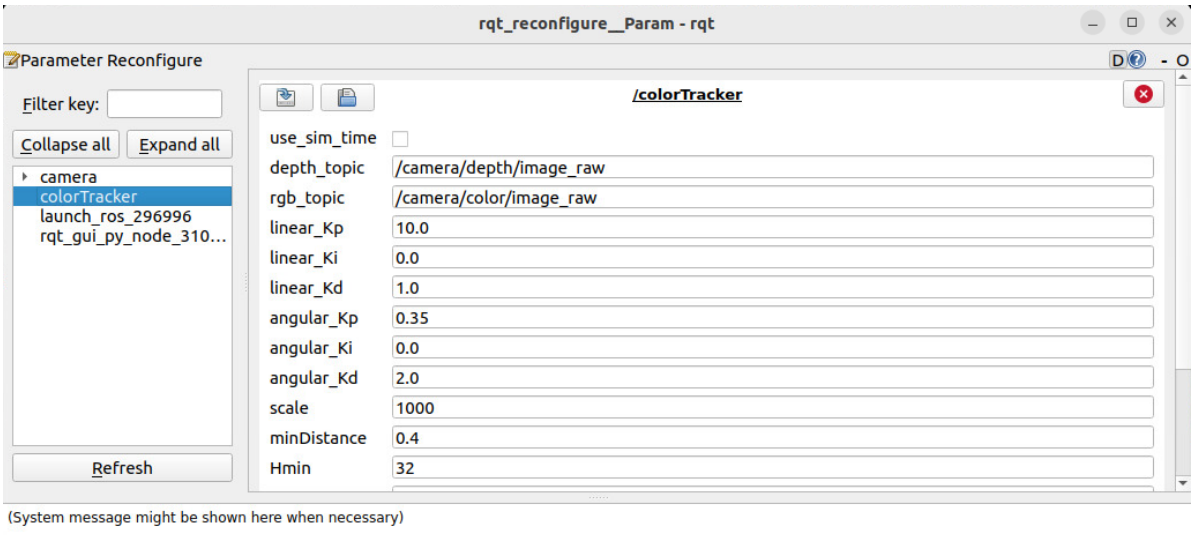
- After selection, the effect is as shown in the figure below:



- Then, press the `spacebar` to enter following mode, slowly move the object, the car will follow the movement. Note that if the depth information of the object is 0, the car will stop, so it is recommended to choose a larger object so that the depth information will be better.

3.1 Dynamic Parameter Adjustment

```
ros2 run rqt_reconfigure rqt_reconfigure
```



- After modifying parameters, click on the blank area of the GUI to write parameter values. Note that it will only take effect for the current startup. For permanent effect, you need to modify the parameters in the source code.

Parameter analysis:

【linear_Kp】, 【linear_Ki】, 【linear_Kd】: Linear speed PID control during car following process.

【angular_Kp】, 【angular_Ki】, 【angular_Kd】: Angular speed PID control during car following process.

【minDistance】: Following distance, always maintain this distance.

【scale】: PID proportional scaling.

【Hmin】, 【Smin】, 【Vmin】: Represent the minimum values of HSV

【Hmax】, 【Smax】, 【Vmax】: Represent the maximum values of HSV

4. Source Code Analysis

`colorTracker.py` source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/colorTracke
r.py
```

This program mainly has the following functions:

- Subscribe to depth camera topics to get depth images;
- Get keyboard and mouse events for switching modes and color picking;
- Process images to get object center coordinates and depth distance information.
- Calculate distance PID and center coordinate PID to get car forward speed distance and turning angle.

Some core code is as follows,

```
# Important functions to get x value, y value and distance_ value
def process(self, rgb_img, action):

#Target center point depth measurement
if self.Center_r > 5:
    points = [
        (int(self.Center_y), int(self.Center_x)),          # Center Point
        (int(self.Center_y + 1), int(self.Center_x +1)),  # Lower right offset
point
        (int(self.Center_y - 1), int(self.Center_x - 1))   # Upper left offset
point
    ]
    valid_depths = []
    for y, x in points:
        depth_val = depth_image_info[y][x]
        if depth_val != 0:
            valid_depths.append(depth_val)
    if valid_depths:
        dist = int(sum(valid_depths) / len(valid_depths))
    else:
        dist = 0
    self.dist = dist
   #When the distance is greater than 0.2 meters and the start state is True,
call the execute method to control the robot movement
   if dist > 0.2 and self.Start_state == True:
    self.execute(self.Center_x, dist)

# According to the x value and y value, use the PID algorithm to calculate the
movement speed and turning angle
def execute(self, rgb_img, action):
    #PID calculation deviation
```

```
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(point_x,320)
    # The car is in the parking area and the speed is 0
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    ...
    # Publish the calculated speed information
    self.pub_cmdVel.publish(twist)
```